

Take-Home Architecture Design Exercise

Context

Caseware builds software that supports audit, assurance, risk, and compliance professionals in delivering high-quality work efficiently and consistently across their firm's practice. Our platforms help practitioners plan engagements, analyze complex financial and operational data, apply professional standards and methodologies, and document defensible conclusions.

AI at Caseware is not intended to replace professional judgment. Instead, it augments practitioners by helping them:

- Investigate and explain findings with traceability and confidence
- Apply consistent approaches across engagements and teams.

This exercise evaluates **architecture, technical judgment, system evolution, and cross-system design** at a Staff level. We are particularly interested in how you identify system boundaries, manage ambiguity, reason about cost, and safely evolve systems that are already running in production.

The Problem

Our customers work in "Engagement Files" which contain all the work that an accounting firm would do for a customer over the course of a year. These engagement files are created based on "Product Templates", which are regularly updated by our content teams in different markets.

Engagement files are initially created with the most recent version of a product template, but those templates receive updates throughout the year. Once a product template has an update published, that update needs to be evaluated by our customers. For each of their engagement files, the customer will either apply the update to an engagement or decline the update.

Users need to see at a glance which of their engagement files have pending updates, and users need a summary of the pending inbound changes in order to make that apply/decline decision. We would like you to design a system that can provide users with that information in an up-to-date manner, for roughly «4,000» firms and «800,000» active

engagement files across «40» products. Engagement counts per firm are heavily skewed: the median firm has «~40», the largest «~40,000». All-in run cost for the feature, inclusive of any inference spend, must stay under «\$X»/month. Product template updates are relatively infrequent, on the order of once per week per product.

Product would like this to be **available in real time** — within seconds of a template being published.

Template versions do **not** form a simple line. Markets branch, versions are occasionally withdrawn after publication, and a firm that declined v5 may later be offered v7, which contains v5's changes. Consider how you handle several updates accumulating before the user makes the accept/decline choice, and what “declined” actually pins.

Actually applying the updated template content is outside the scope of this assignment.

Systems You Can Build On

All versions of all product templates are stored in a database, and individual templates can be quickly retrieved given a template ID and a version. Publishing an updated product template involves writing to this database. Creating a new engagement file involves reading from this database in order to retrieve the template file to use to create the new engagement file.

Product templates are essentially zip archives of structured JSON data. You can assume that you have access to a quick and reliable way to compare two different versions of the same Product Template and extract a JSON diff. However, the users are non-technical and would strongly prefer something human-readable.

Notably, the product template database is shared between customer firms, as they all use the same product templates. This database does not currently retain any information about Engagement files, which are stored in their own customer-specific databases.

Engagement content is client-confidential audit workpaper material, and some firms are contractually or legally required to keep their data in-region («EU», «Canada»). Product template content is not firm-specific.

Engagement files currently store the template ID and version used for their creation. However, the stored form of an engagement is not a directly queryable record of its current state — opening one rehydrates it into memory in the pods serving that session, where volatile financial fields are established. Reading the template information therefore requires loading the engagement, which is a slow operation (~1 minute per file; this should

be treated as a hard constraint). The same system which loads engagements is also responsible for handling engagement creation, and for processing the user's accept/decline decision.

All of these systems are already live. There is no maintenance window, and the pending-update indicator needs to work for **existing** engagements, not only newly created ones.

You may assume you can add hooks or events to any actions taken by either the product template storage system, or by the engagement management system.

Part 1: Architecture & Design (Primary Focus)

Please provide a short design document (2-4 pages max; diagrams don't count toward the limit) that covers:

1. **Architecture and data ownership**
2. **Correctness and production evolution**
3. **Scale, cost and operational characteristics** — show your arithmetic against the numbers above
4. **Generation and evaluation of the human-readable summaries**
5. **Key tradeoffs, assumptions, and the one requirement above you would challenge**

Address migration and backfill, unreliable event delivery, observability and SLOs, data residency, and failure recovery where they matter to your design. On (4), note that a summary shown to a user in March may need to be defended during a regulatory review in November.

Please also tell us which part of your design is riskiest to get wrong, and what you deliberately chose to leave out.

Part 2: Targeted Implementation (Required)

Implement **one** narrow component in **Java**: the template-publish fan-out worker. A single publish may require processing a large number of engagements, and the downstream call you depend on takes ~1 minute per engagement and is owned by another team with limited capacity.

Guidelines:

- Demonstrate appropriate handling of concurrency, failure, idempotency and downstream capacity
- Focus on interfaces, contracts and correctness; tests or validation logic are a plus

This is **not** a full application build.

In a short README, explain your key implementation tradeoffs.

AI Usage (Important)

You may use **any AI tools you normally use** (e.g., ChatGPT, Copilot, Claude).

During the follow-up review, we will ask you to explain:

- Where AI helped you
- Where you corrected or ignored AI output
- How you would guide other engineers using AI on this system
- Where AI should *not* be trusted in this domain

We are evaluating **judgment and maturity**, not just usage.

Assumptions & Constraints

- You may choose any cloud provider or stack (note we are an AWS shop)
- You may state reasonable assumptions
- Optimize for clarity and tradeoffs over completeness

Deliverables

Please submit:

1. Design document (PDF or Markdown)
2. Code for Part 2, with a short README
3. (Optional) Session History for any AI tooling used
4. Any diagrams you created

What We're Evaluating

- Architecture and system design judgment
- Understanding of LLMs and other AI tools
- Reasoning about cost, scale, and safe evolution of live production systems
- Clarity of communication
- Thoughtful use of AI in the Software Development Process

Time Expectation

We expect this exercise to take **no more than 4 hours total**.

Please do not over-optimize.

We would rather see a short document with sharp decisions than a long one that hedges.

Follow-Up

We will review your submission. If we like it, you will be invited to a **live design review** to discuss decisions, tradeoffs, and proposed implementation of the system.